Desenvolvimento de Sistemas para Dispositivos Móveis

Listas, formulários e Material 3

DIM0524 — Turma 01 · Sprint 1 · 14/09

Prof. Fernando · UFRN · 2026.2

Roteiro da semana

Segunda, 14/09

Bloco	O que vemos
Ambiente	Android Studio 2025.3.2+ ou Codespaces
Fecha a Sprint 0	`@Composable`, estado, componentes, Material 3
Abre a Sprint 1	Listas com `LazyColumn`, formulários e validação
Tema	Material 3, claro e escuro

Quarta, 16/09 — Acompanhamento online · 🚀 **Entrega da Sprint 0, 23:59**
Segunda, 21/09 — Responsividade, acessibilidade, navegação e testes

> Hoje construímos uma tela do zero, em passos.

Android Studio

O exemplo usa AGP 9.1, que exige o **Android Studio Panda 2 (2025.3.2) ou mais novo**. O do laboratório (Ladybug, 2024) **não abre** o projeto; até a atualização, use o Codespaces.

	Codespaces (navegador)	Android Studio local (Panda 2+)
`@Preview`	Android Studio no Codespace: `mise run studio`	funciona
Hot Reload	janela de celular: `mise run app:celular`	funciona
Emulador Android	não: gere o APK e instale no emulador	sim

> Tudo aparece na porta 6080 do Codespace (área de trabalho no navegador, senha `vscode`). Passo a passo: `exemplos/tarefas-compose/PASSOS.md`.

Como rodar

- **Android Studio (local ou no Codespace):** run configuration `composeApp` → Run ▸, ou o `@Preview` em **Split**.
- **Desktop com Hot Reload:** `./gradlew :composeApp:hotRunDesktop --auto` (ou `mise run app`): cada salvamento recarrega a tela.

A UI compartilhada fica em `composeApp/src/commonMain/kotlin`. A mesma `App()` roda no Android e no Desktop. O `@Preview` mostra a tela sem rodar o app.

Onde paramos

Na Sprint 0 vimos os fundamentos:

```
KMP: módulos shared/app e a regra de dependência
Compose: @Composable, recomposição, estado
Componentes próprios · Material 3 · Modificadores
```

Hoje concluímos o que faltou e damos o passo seguinte: uma tela com lista e formulário.

Recap: os 3 conceitos que sustentam tudo

Conceito	Em uma frase
`@Composable`	Função que descreve a UI a partir de dados
Recomposição	Mudou o estado → o Compose redesenha o que depende dele
Estado elevado	O estado sobe; o componente recebe dados e devolve eventos

```
var contador by remember { mutableStateOf(0) }
Button(onClick = { contador++ }) { Text("Cliquei $contador") }
```

> `remember { mutableStateOf(...) }` guarda o estado entre recomposições.

Entendendo: lambdas, `{ }` e `it`

Compose usa muito `{ }`. Quase sempre é uma lambda — um bloco de código passado como argumento.

```
Button(onClick = { contador++ }) { Text("Cliquei") }
//             └── lambda: o que fazer no clique
//                                  └── trailing lambda: o conteúdo do botão
```

- `() -> Unit` é o tipo de uma função sem parâmetros que nada retorna (`Unit` ≈ `void`).
- Se a lambda é o último argumento, ela sai dos parênteses (*trailing lambda*) — daí `Column { }`, `Card { }`, `LazyColumn { }`.
- `it` é o nome automático do parâmetro único: em `onValueChange = { texto = it }`, o `it` é o novo texto.

> **Kotlin para quem conhece Java:** a lambda é como a do Java (`() -> {}`); o Kotlin deixa tirá-la dos parênteses e usar `it`. É o que dá o visual de DSL do Compose.

📖 **Ref.** Kotlin — lambdas · trailing lambdas

Entendendo: estado com `by`

```
var contador by remember { mutableStateOf(0) }
```

Três peças numa linha só:

- `mutableStateOf(0)` — cria um estado observável: quando muda, o Compose recompõe quem o lê.
- `remember { }` — guarda esse estado entre recomposições (sem ele, voltaria a 0 a cada redesenho).
- `by` — propriedade delegada: ler `contador` devolve o valor; `contador = x` atualiza o estado. Sem o `by`, seria `contador.value`.

> **Kotlin para quem conhece Java:** o `by` delega leitura e escrita a outro objeto (aqui, o estado). Em Java você chamaria `get()/set()`; o `by` faz `contador` parecer uma variável comum.

📖 **Ref.** Kotlin — delegated properties · Compose — state

Entendendo: `data class`

```
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)
```

- Uma classe para guardar dados. O `data` gera `equals`, `hashCode`, `toString` e `copy()`.
- `val` é imutável (como `final`); `var` é mutável. Aqui é tudo `val`.
- `= false` é um valor padrão: dá para criar `Tarefa(1, "X")` sem informar `feita`.
- `copy()` cria uma cópia mudando um campo: `tarefa.copy(feita = true)`.

> **Kotlin para quem conhece Java:** é como um `record`, com o acréscimo do `copy()`, usado para atualizar estado imutável — `lista.map { it.copy(...) }`.

📖 **Ref.** Kotlin — data classes

Entendendo: por que `val` e `copy`?

A tela é função do estado. O que muda é o estado (um `var` observável); os dados que ele guarda são imutáveis (`val`).

```
var tarefas by remember { mutableStateOf(listOf<Tarefa>()) }

// map percorre a lista e devolve uma lista NOVA:
tarefas = tarefas.map {
  if (it.id == alvo)             // it = tarefa da vez; é a alvo?
    it.copy(feita = !it.feita)   // sim: cópia com 'feita' invertido
  else
    it                           // não: a mesma tarefa, sem mudar
}
// atribuir de volta a 'tarefas' dispara a recomposição
```

- Só a tarefa-alvo muda; as outras voltam como `it` (mesmos objetos). Com `key`, o Compose redesenha só o item alterado.
- Nada é mutado no lugar: `map` e `copy` criam novos; a lista antiga fica intacta.

> "Por que não `var`?" Mutar um campo por dentro não avisa o Compose. O padrão é dado imutável e reatribuir o estado.

📖 **Ref.** Compose — state · Kotlin — data classes

O exemplo de hoje: Tarefas

Uma tela construída em 5 passos — cada um roda:

Passo	O que entra
1	Um `@Composable` com estado
2	Componente próprio reutilizável (`CartaoTarefa`)
3	Lista com `LazyColumn`
4	Formulário com validação
5	Material 3 e tema (claro/escuro)

Código pronto para consulta: `exemplos/tarefas-compose/` (ver `PASSOS.md`).

Passo 1

Um `@Composable` com estado

Passo 1 — a primeira tela

```
@Composable
fun App() {
  MaterialTheme {
    var contador by remember { mutableStateOf(0) }
    Button(onClick = { contador++ }) {
      Text("Cliquei $contador")
    }
  }
}
```

- `App()` é o ponto de entrada da UI — o que o `MainActivity` monta.
- `remember { mutableStateOf(0) }` guarda o estado; clicar recompõe o texto.

> Ponto de partida da tela: uma função `@Composable` com estado. Nos próximos passos ela vira uma lista de tarefas.

📖 **Ref.** Compose — state

Passo 2

Componente próprio, estado elevado

Passo 2 — `CartaoTarefa`

```
data class Tarefa(val id: Int, val titulo: String, val feita: Boolean = false)

@Composable
fun CartaoTarefa(tarefa: Tarefa, onAlternar: () -> Unit) {
  Card(Modifier.fillMaxWidth()) {
    Row(Modifier.padding(12.dp), verticalAlignment = Alignment.CenterVertically) {
      Checkbox(checked = tarefa.feita, onCheckedChange = { onAlternar() })
      Spacer(Modifier.width(8.dp))
      Text(tarefa.titulo)
    }
  }
}
```

Estado elevado: o cartão recebe a `tarefa` e devolve o evento `onAlternar`, sem guardar estado interno. Fica reutilizável e testável.

📖 **Ref.** Compose — state e state hoisting

Passo 3

Lista com `LazyColumn`

Passo 3 — `LazyColumn`

```
LazyColumn(verticalArrangement = Arrangement.spacedBy(8.dp)) {
  items(tarefas, key = { it.id }) { tarefa ->  // tarefa = a linha clicada
    CartaoTarefa(tarefa) {                     // ação ao tocar no checkbox
      tarefas = tarefas.map {                  // it = cada item da lista
        if (it.id == tarefa.id)                // é a linha clicada?
          it.copy(feita = !it.feita)           // sim: cópia com 'feita' invertido
        else it                                // não: mantém o item
      }
    }
  }
}
```

- `LazyColumn` cria só os itens visíveis — bom para listas grandes.
- `tarefa` vem do `items`; `it` é cada item dentro do `map`.

> Uma `Column` com `verticalScroll` cria tudo de uma vez e pode apresentar problemas em performance.
> Para listas que podem ser grandes, melhor usar o `LazyColumn` que renderiza apenas o que é necessário.

📖 **Ref.** Compose — lazy lists

Entendendo: `key` na LazyColumn

```
items(tarefas, key = { it.id }) { tarefa ->
  CartaoTarefa(tarefa) { /* ... */ }
}
```

- `key = { it.id }` dá a cada item uma identidade estável (o `id`), única na lista.
- Sem `key`, o Compose identifica os itens pela posição (índice). Se a lista muda — inserir, remover, reordenar — os itens trocam de posição e o estado de uma linha pode ir parar em outra.
- Com `key`, o Compose casa cada linha pelo `id`: preserva o estado das que ficaram e recompõe só a que mudou.

> Exemplo: remova a 1ª tarefa. Sem `key`, a 2ª herda o estado da 1ª (rolagem, animação); com `key`, cada uma mantém o seu. Use `key` sempre que a lista puder mudar.

📖 **Ref.** Compose — lazy lists (chaves de item)

Passo 4

Formulário com validação

Passo 4 — formulário

```
var texto by remember { mutableStateOf("") }
val valido = texto.isNotBlank()

Row(verticalAlignment = Alignment.CenterVertically) {
  OutlinedTextField(
    value = texto,
    onValueChange = { texto = it },
    label = { Text("Nova tarefa") },
    isError = texto.isNotEmpty() && !valido,
    modifier = Modifier.weight(1f),
  )
  Button(onClick = { /* adiciona */ }, enabled = valido) { Text("Adicionar") }
}
```

A validação vem do estado (`valido`). O botão habilita quando o título é válido, e o campo sinaliza erro.

📖 **Ref.** Compose — campos de texto · Compose — state · Documentação do componente de UI

Passo 5

Material 3 e tema

Passo 5 — Material 3

Os componentes (`Button`, `Card`, `OutlinedTextField`) já seguem o Material 3. O tema define cor e tipografia:

```
MaterialTheme(
  colorScheme = if (isSystemInDarkTheme()) darkColorScheme() else lightColorScheme()
) {
  App()
}
```

- Cor e tipografia vêm do `MaterialTheme`, não fixadas à mão.
- Modo claro e escuro com pouca configuração.

> No emulador, alterne o tema do sistema e veja a tela responder, sem mudar uma linha da `App()`.

📖 **Ref.** Material 3 no Compose

`@Preview` — ver sem rodar

Em `commonMain`, anote uma função sem parâmetros:

```
import androidx.compose.ui.tooling.preview.Preview   // a antiga, `org.jetbrains...`, não compila

@Preview
@Composable
fun AppPreview() { App() }
```

O Android Studio renderiza a tela ao lado do código.

> No ambiente local, o preview responde na hora — sem esperar o emulador.

📖 **Ref.** Compose — @Preview

Boas práticas de hoje

Prática	Por quê
Estado elevado	Componentes reutilizáveis e testáveis
Componente sem estado interno	Recebe dados, devolve eventos
`key` em listas	Rolagem e desempenho estáveis
Cor/tipografia via `MaterialTheme`	Tema consistente, claro e escuro

Para a próxima aula (21/09)

- **Responsividade e adaptatividade**: classes de tamanho de janela
- **Acessibilidade**: descrições, contraste, alvos de toque
- **Navegação**: Navigation Compose — rotas tipadas, argumentos, deep links
- **Testes** em Compose Multiplatform

No dia 16/09 é o acompanhamento online de projeto. 🚀 Entrega da Sprint 0 nesse dia, 23:59.

> Lembrete: a primeira tela da Sprint 0 agora é opcional, mas com o que vimos hoje vocês já podem construí-la.

Onde estudar depois

Fonte	Foco
`kotlinlang.org/docs/multiplatform` · *Compose Multiplatform*	Criar o primeiro app, alvos e execução
`developer.android.com/develop/ui/compose`	Compose e listas
`m3.material.io`	Material 3: cor, tipografia, componentes